

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)

Analysis of Business Analytics - I Practical

Practical - VIII

Roll No. : T048	Name: Lavkush Vishwakarma
Class : TYIT	Date/Time of Submission : 11/09/2026

Aim: Confidence Interval Estimation:

A) Confidence Interval Estimation

Dataset Information

- **Dataset size:** 1000 rows $\times$ 17 columns
- **Revenue observations:** 1000
- **Missing revenue values:** 0
- **Sample mean revenue:** 322.9667
- **Sample standard deviation:** 245.8853

Confidence Intervals for Mean Revenue

Confidence Level	Lower Bound	Upper Bound
90%	310.1652	335.7683
95%	307.7084	338.2251
99%	302.8999	343.0336

As expected, the interval becomes **wider as the confidence level increases**.

Customer Type

Customer Type	Count	Proportion
Member	501	50.1%
Normal	499	49.9%

95% CI for proportion of Member customers:

47.00% to 53.20%

So, based on this sample, the population proportion of Member customers is plausibly between **47.00% and 53.20%** at the 95% confidence level.

B) Bootstrapping

Using **1,000 bootstrap samples** with replacement:

- **Original sample median revenue:** 253.848
- **Bootstrap 2.5th percentile:** 232.869
- **Bootstrap 97.5th percentile:** 273.614
- **95% Bootstrap CI:** 232.869 to 273.614

The original median (**253.848**) lies comfortably inside the bootstrap confidence interval.

COMPLETE SOURCE CODE:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# =====
# PART A: CONFIDENCE INTERVAL ESTIMATION
# =====

# 1. Load dataset
df = pd.read_csv("supermarket_sales.csv")
# 2. Display first five records and dataset size
print("First Five Records:")

print(df.head())
print("\nDataset Size:")
print(df.shape)
# 3. Select revenue column
revenue = df["revenue"]
# 4. Check and handle missing values
print("\nMissing Values in Revenue:",
revenue.isnull().sum())
# Remove missing values
revenue = revenue.dropna()
# 5. Calculate sample size, mean and standard deviation
```

```

n = len(revenue)
sample_mean = revenue.mean()
sample_std = revenue.std(ddof=1)
print("\nSample Size:", n)
print("Sample Mean:", sample_mean)
print("Sample Standard Deviation:", sample_std)
# 6, 7, 8. Calculate confidence intervals
confidence_levels = [0.90, 0.95, 0.99]
results = []
for confidence in confidence_levels:
    alpha = 1 - confidence
    t_critical = stats.t.ppf(
        1 - alpha / 2,
        df=n-1
    )
    margin_error = (
        t_critical * sample_std / np.sqrt(n)
    )
    lower = sample_mean - margin_error
    upper = sample_mean + margin_error
    results.append([
        f"{confidence*100:.0f}%",
        lower,
        upper
    ])
# 9. Display confidence intervals in table
ci_table = pd.DataFrame(
    results,
    columns=["Confidence Level", "Lower Bound", "Upper Bound"]
)
print("\nConfidence Intervals:")
print(ci_table.to_string(index=False))
# 10. Compare confidence intervals
print("\nComparison:")
print("Higher confidence levels produce wider confidence intervals.")
# 11. Visualization
levels = [90, 95, 99]
lower_bounds = [x[1] for x in results]
upper_bounds = [x[2] for x in results]
errors_lower = [
    sample_mean - x for x in lower_bounds
]
errors_upper = [
    x - sample_mean for x in upper_bounds
]
plt.figure(figsize=(10, 6))
plt.errorbar(
    levels,
    [sample_mean] * 3,
    yerr=[errors_lower, errors_upper],
    fmt='o',
    capsize=6
)
plt.axhline(
    sample_mean,
    linestyle='--',

```

```

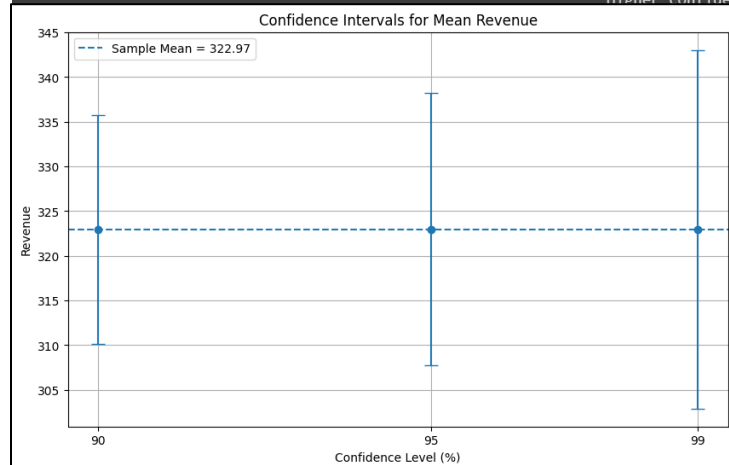
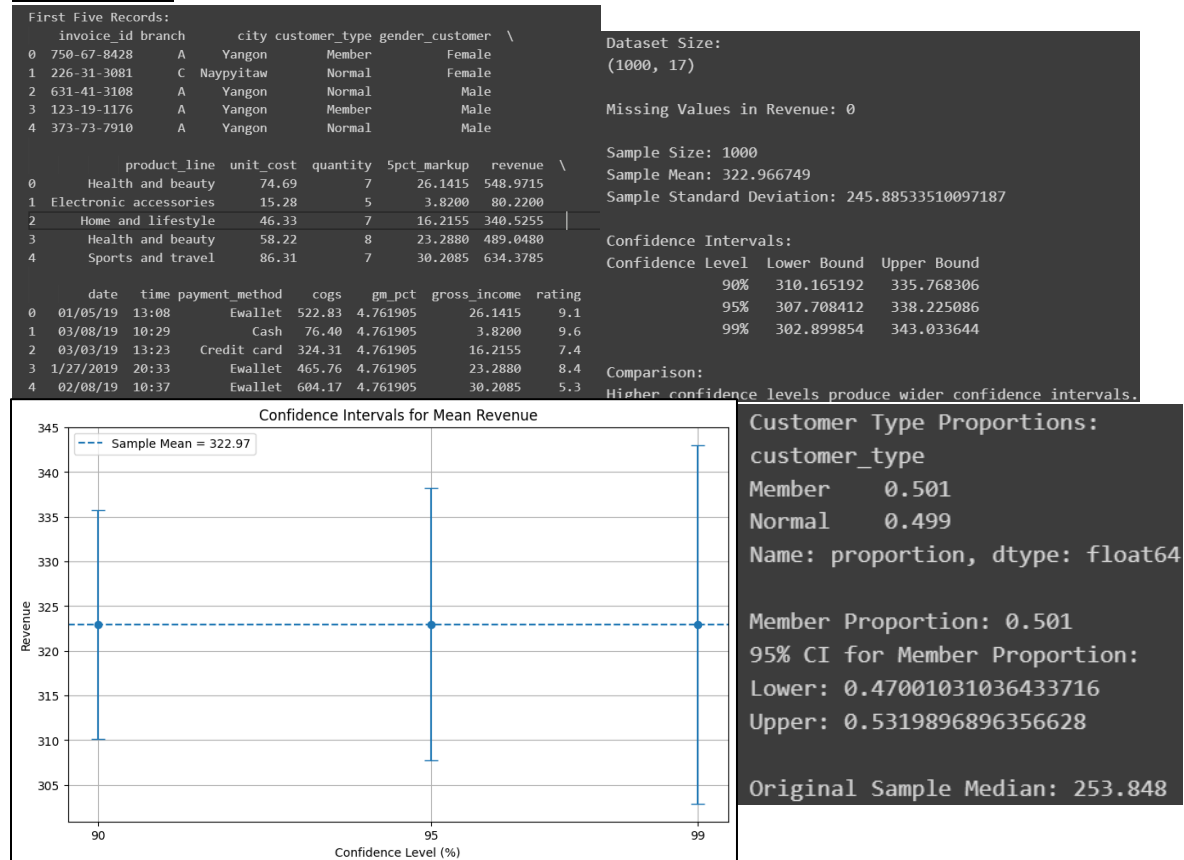
    label=f"Sample Mean = {sample_mean:.2f}"
)
plt.xlabel("Confidence Level (%)")
plt.ylabel("Revenue")
plt.title("Confidence Intervals for Mean Revenue")
plt.xticks(levels)
plt.legend()
plt.grid(True)
plt.show()
# =====
# CUSTOMER TYPE PROPORTION
# =====
# 13. Calculate proportion of each customer type
customer_proportion = df["customer_type"].value_counts(
    normalize=True
)
print("\nCustomer Type Proportions:")
print(customer_proportion)
# Number of Member customers
member_count = (
    df["customer_type"] == "Member"
).sum()
total_customers = df["customer_type"].notna().sum()
member_proportion = member_count / total_customers
# 14. 95% CI for Member proportion
z_critical = stats.norm.ppf(0.975)
margin = z_critical * np.sqrt(
    member_proportion *
    (1 - member_proportion) /
    total_customers
)
lower_prop = member_proportion - margin
upper_prop = member_proportion + margin
print("\nMember Proportion:", member_proportion)
print("95% CI for Member Proportion:")
print("Lower:", lower_prop)
print("Upper:", upper_prop)
# =====
# PART B: BOOTSTRAPPING
# =====
# 1. Original sample median
original_median = np.median(revenue)
print("\nOriginal Sample Median:", original_median)
# 2-4. Bootstrap sampling
np.random.seed(42)
bootstrap_medians = []
for i in range(1000):
    bootstrap_sample = np.random.choice(
        revenue,
        size=len(revenue),
        replace=True
    )
# 3. Calculate bootstrap median
bootstrap_median = np.median(
    bootstrap_sample
)
bootstrap_medians.append(
    bootstrap_median

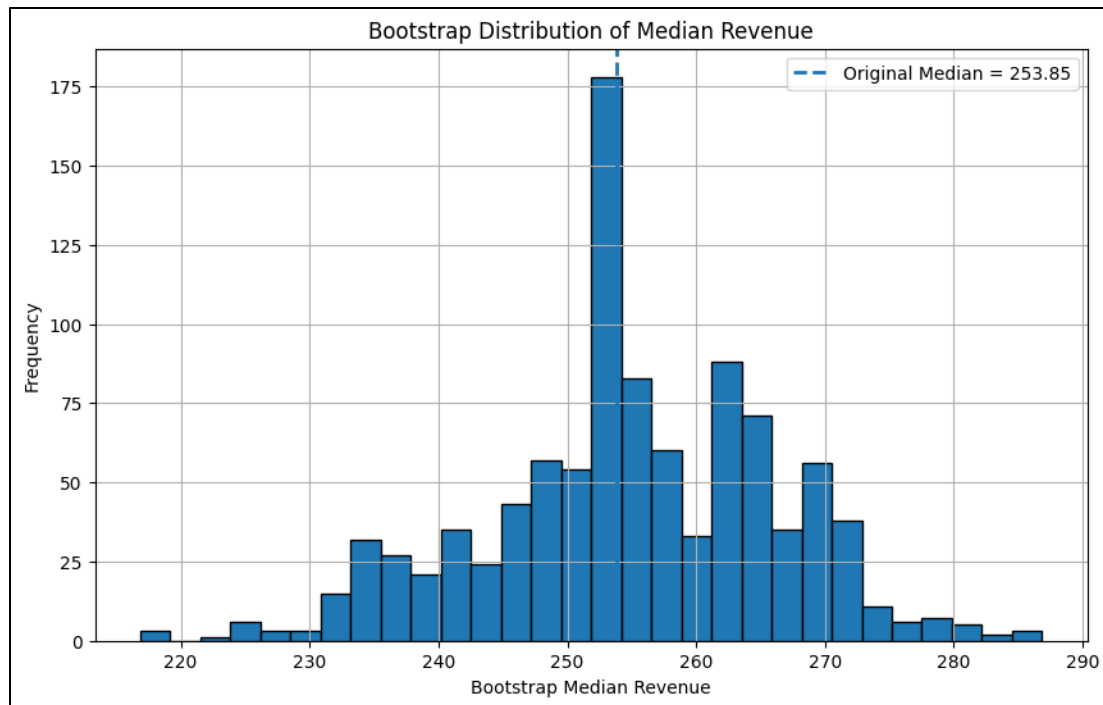
```

```

)
bootstrap_medians = np.array(
    bootstrap_medians
)
# 5. Histogram
plt.figure(figsize=(10, 6))
plt.hist(
    bootstrap_medians,
    bins=30,
    edgecolor="black"
)
plt.axvline(
    original_median,
    linestyle="--",
    linewidth=2,
    label=f"Original Median = {original_median:.2f}"
)
plt.xlabel("Bootstrap Median Revenue")
plt.ylabel("Frequency")
plt.title("Bootstrap Distribution of Median Revenue")
plt.legend()
plt.grid(True)
plt.show()
# 6. Calculate percentiles
lower_bootstrap = np.percentile(
    bootstrap_medians,
    2.5
)
upper_bootstrap = np.percentile(
    bootstrap_medians,
    97.5
)
# 7. 95% bootstrap confidence interval
print("\n95% Bootstrap Confidence Interval:")
print("Lower Bound:", lower_bootstrap)
print("Upper Bound:", upper_bootstrap)
# 8. Display original median and CI
print("\nOriginal Sample Median:", original_median)
print(
    f"95% Bootstrap CI: "
    f"({lower_bootstrap:.3f}, "
    f"{upper_bootstrap:.3f})"
)
# 9. Compare
if lower_bootstrap <= original_median <= upper_bootstrap:
    print(
        "\nThe original median lies within "
        "the bootstrap confidence interval."
    )
else:
    print(
        "\nThe original median lies outside "
        "the bootstrap confidence interval."
    )

```

OUTPUT:-



95% Bootstrap Confidence Interval:

Lower Bound: 232.869

Upper Bound: 273.61372500000004

Original Sample Median: 253.848

95% Bootstrap CI: (232.869, 273.614)

The original median lies within the bootstrap confidence interval.

Interpretations

95% Mean Confidence Interval:

The 95% confidence interval for the population mean supermarket revenue is approximately **307.71 to 338.23**. This means that, using this sampling method, we are 95% confident that the true population mean revenue lies within this range.

Member Customer Proportion:

The estimated proportion of Member customers is **50.1%**, with a 95% confidence interval of approximately **47.00%–53.20%**. This suggests that Member customers make up roughly half of the supermarket's customer base.

Bootstrap Median:

The original sample median revenue is **253.848**, while the 95% bootstrap confidence interval is approximately **232.869–273.614**. Since the original median falls within this interval, the sample median provides a reasonable estimate of the population median.

Conclusion:

Confidence intervals provide businesses with a range of plausible values for population parameters such as mean revenue and customer proportions. Bootstrapping is particularly useful for estimating the uncertainty of statistics such as the median without relying heavily on assumptions about the population distribution. Together, these techniques help supermarkets make more reliable decisions from sample data rather than relying only on single-point estimates.